

Week 14 Assignment

Ronish Shrestha

Part 1 — Create the Project

Create a new folder, open it in VS Code, and run the following commands:

```
mkdir week14-advanced-api
cd week14-advanced-api
npm init -y
npm install express mongoose
```

Part 2 — Project Structure

Create the following files and folders inside the project:

```
week14-advanced-api/
├── controllers/
│   └── userController.js
├── models/
│   └── userModel.js
├── routes/
│   └── userRoutes.js
└── server.js
```

Part 3 — server.js

This file sets up the Express server and connects to MongoDB.

Explanation

- Imports Express and Mongoose
- Mounts user routes at /api/users
- Uses express.json() middleware to parse request bodies
- Connects to MongoDB and logs success or error
- Starts the server on port 5000

Code

```
const express = require('express');
const mongoose = require('mongoose');
const userRoutes = require('./routes/userRoutes');

const app = express();

app.use(express.json());
```

```

app.use('/api/users', userRoutes);

const MONGO_URI = 'mongodb+srv://<user>:<pass>@cluster0.mongodb.net/week14db';

mongoose
  .connect(MONGO_URI)
  .then(() => console.log('Connected to MongoDB successfully'))
  .catch((err) => console.error('MongoDB connection error:', err));

const PORT = 5000;
app.listen(PORT, () => {
  console.log(`Server running on http://localhost:${PORT}`);
});

```

Part 4 — User Model (userModel.js)

Defines the schema for storing user data in MongoDB.

Explanation

- name — required string field
- email — required, unique string field
- age — optional number field
- timestamps: true adds createdAt and updatedAt automatically

Code

```

const mongoose = require('mongoose');

const userSchema = new mongoose.Schema(
  {
    name: {
      type: String,
      required: [true, 'Name is required'],
      trim: true,
    },
    email: {
      type: String,
      required: [true, 'Email is required'],
      unique: true,
      lowercase: true,
      match: [/^\S+@\S+\.\S+$/, 'Please enter a valid email'],
    },
    age: {
      type: Number,
      min: [0, 'Age cannot be negative'],
      max: [120, 'Age must be 120 or below'],
    },
  },
  { timestamps: true }
);

const User = mongoose.model('User', userSchema);
module.exports = User;

```

Part 5–9 — Controller (UserController.js)

Contains all CRUD logic: createUser, getUsers (with filtering and pagination), updateUser, and deleteUser.

createUser (Part 6)

- Accepts a single object or an array in the request body
- Uses insertMany to bulk-create users in one operation

```
const createUser = async (req, res) => {
  try {
    const data = req.body;
    let savedUsers;
    if (Array.isArray(data)) {
      savedUsers = await User.insertMany(data);
    } else {
      const user = new User(data);
      savedUsers = await user.save();
    }
    res.status(201).json({ message: 'User(s) created', data: savedUsers });
  } catch (error) {
    res.status(400).json({ message: 'Error', error: error.message });
  }
};
```

getUsers (Part 7)

- Supports ?name= for case-insensitive filtering using \$regex
- Supports ?page= and ?limit= for pagination using skip() and limit()

```
const getUsers = async (req, res) => {
  try {
    const { name, page = 1, limit = 10 } = req.query;
    const filter = {};
    if (name) {
      filter.name = { $regex: name, $options: 'i' };
    }
    const pageNum = parseInt(page);
    const limitNum = parseInt(limit);
    const skip = (pageNum - 1) * limitNum;
    const users = await User.find(filter).skip(skip).limit(limitNum);
    const total = await User.countDocuments(filter);
    res.status(200).json({
      message: 'Users retrieved successfully',
      total,
      page: pageNum,
      totalPages: Math.ceil(total / limitNum),
      data: users,
    });
  } catch (error) {
    res.status(500).json({ message: 'Error', error: error.message });
  }
};
```

updateUser (Part 8)

- Finds user by ID from the URL parameter and updates with request body
- Returns 404 if user not found

```
const updateUser = async (req, res) => {
  try {
    const { id } = req.params;
    const updates = req.body;
    const updatedUser = await User.findByIdAndUpdate(id, updates, {
      new: true,
      runValidators: true,
    });
    if (!updatedUser) {
      return res.status(404).json({ message: 'User not found' });
    }
    res.status(200).json({ message: 'User updated', data: updatedUser });
  } catch (error) {
    res.status(400).json({ message: 'Error', error: error.message });
  }
};
```

deleteUser (Part 9)

- Deletes user by ID
- Returns success message or 404 if not found

```
const deleteUser = async (req, res) => {
  try {
    const { id } = req.params;
    const deletedUser = await User.findByIdAndDelete(id);
    if (!deletedUser) {
      return res.status(404).json({ message: 'User not found' });
    }
    res.status(200).json({ message: 'User deleted', data: deletedUser });
  } catch (error) {
    res.status(500).json({ message: 'Error', error: error.message });
  }
};

module.exports = { createUser, getUsers, updateUser, deleteUser };
```

Part 10 — Routes (userRoutes.js)

Maps HTTP methods and URL patterns to the controller functions.

Route explanation

- POST /api/users → createUser
- GET /api/users → getUsers (with optional query params)
- PUT /api/users/:id → updateUser
- DELETE /api/users/:id → deleteUser

Code

```
const express      = require('express');
const router       = express.Router();
const { createUser, getUsers,
      updateUser, deleteUser } = require('../controllers/userController');

router.post  ('/',      createUser);
router.get   ('/',      getUsers);
router.put  ('/:id',    updateUser);
router.delete('/:id',   deleteUser);

module.exports = router;
```

Part 11 — Import Routes into server.js

Add the import and mount it in server.js (already shown above in Part 3):

```
const userRoutes = require('./routes/userRoutes');
app.use('/api/users', userRoutes);
```

Part 12 — Testing with Postman

Start the server

```
node server.js

# MongoDB connected successfully
# Server running on port 5000
```

1. Create Users — POST /api/users

Body (raw JSON):

```
[
  { "name": "Alice", "email": "alice@example.com", "age": 25 },
  { "name": "Bob",   "email": "bob@example.com",   "age": 30 },
  { "name": "Carol", "email": "carol@example.com", "age": 28 },
  { "name": "David", "email": "david@example.com", "age": 35 },
  { "name": "Eve",   "email": "eve@example.com",   "age": 22 }
]
```

Multiple users are created at once using insertMany.

2. Get All Users — GET /api/users

```
GET http://localhost:5000/api/users
```

3. Filter Users by Name — GET /api/users?name=alice

```
GET http://localhost:5000/api/users?name=alice
```

4. Paginate Users — GET /api/users?page=1&limit=3

```
GET http://localhost:5000/api/users?page=1&limit=3
```

5. Update User — PUT /api/users/:id

Body:

```
{
  "name": "Alice Updated",
  "age": 26
}
```

6. Delete User — DELETE /api/users/:id

```
DELETE http://localhost:5000/api/users/<user_id>
```

Deletes the user matching the provided MongoDB ObjectId.